Cache Simulation

David Floyd, Stanley Barnes

***Abstract*—The objective of this project is to show the behavior of cache systems and performance differences between different optimization techniques. A caching system was modeled in python that stores data addresses in a deque. The cache removed the least recently used elements once full. Additionally an option was added to enable prefetching in which, upon an access, the next adjacent address is also cached. The performance was measured and graphed for easy evaluation.**

***Index Terms*—Cache, LRU, Prefetching, Queue.**

I. Introduction (*Heading 1*)

Modern day CPUs are incredibly fast, due to many reasons. One may be led to believe that faster clocks or more transistors contribute to the largest speedup, but in reality, the development of high speed caches have improved the performance of CPU tremendously. For example, reading directly from memory might take 80 times longer than reading from the cache. Since caches are fundamental to the modern processor, this lab sought to simulate the operation of a cache. Thus, a program was written in Python to support reading of a memory file, and output the misses, hits, accesses, and hit rate. When the user runs the program, they have the option to specify the amount of sets in the cache, list the number of blocks per set, the size of the blocks in bytes, and which memory file to read from. Additionally, the simulator also implements a prefetching operation to grab the N+1 index to increase hit rate.

II. Design Specifications

In order to implement the cache structure, a deque was implemented. This was chosen due to the ease of implementing the Least Recently Used (LRU) logic that was desired. By setting the max length of the deque to the block count, it can be easily ensured that the size of the queue will be set and not exceed. Furthermore, this structure stores data such that the oldest element will be stored on the left, meaning that by popping from the left side of the structure will automatically remove the least recently used element. In order to ensure that this logic will work cleanly, one additional piece of logic was necessary. Upon a cache hit, that element is removed from the deque and then re-added. This moves it to the right of the deque, effectively setting it as the most recently used element.

Our design is controlled from the command line by command line arguments passed as parameters to the main execution of the program and parsed using the *argparse* library. Passing the *--sets* tag followed by an integer allows the caller to set the number of sets that will be used in the cache. *--blocks* tag followed by an integer, sets the associativity, or number of blocks per set. *--size* followed by an integer sets the size of the blocks, as a number of bytes. *--trace* should be passed with a trace file path name, this is where the program will look for the addresses to cache. Finally the *--prefetch* tag is passed by itself and when present enables prefetching in the cache, if this is not passed, the cache will not implement prefetching.

III. Verification

In order to prove that our cache simulation logic was correct, small sets of addresses were calculated by hand and manually added to a cache. Working though sets of addresses and different cache types gave us verified data that we used to check our simulation against. By showing that the simulated values (I.E. hits, misses, miss, miss rate) matched the hand calculated values it can be shown that the simulated cache is an accurate model and effectively implements the caching logic expected.

To verify, we used question 5.2.2 from *Computer Organization and Design* [1, p. 498]. The question gives a list of memory addresses, and asks to find the hits, misses, and hit rate for the problem. Specifically, it asks for a cache of eight sets, one block per set, and two bytes per word. Below is the hand calculated:

Table I: Hand Calculated Cache Simulation

Address (hex)	Address (Binary)	Tag	Index	Offset	Hit/Miss
0x03	0000 0011	0000	001	1	Miss
0xb4	1011 0100	1011	010	0	Miss
0x2b	0010 1011	0010	101	1	Hit
0x02	0000 0010	0000	001	0	Miss
0xbf	1011 1111	1011	111	1	Miss
0x58	0101 1000	0101	100	0	Miss
0xbe	1011 1110	1011	111	0	Hit
0x0e	0000 1110	0000	111	0	Miss
0xb5	1011 0101	1011	010	1	Hit
0x2c	0010 1100	0010	110	0	Miss
0xba	1011 1010	1011	101	0	Miss
0xfd	1111 1101	1111	110	1	Miss

```
python3 cache_sim.py --sets 8 --blocks 1 --size 2
--trace  test.txt
```

Using the command above for our program gives the following results:

Accesses:	12
Hits:	3
Misses:	9
Miss Rate:	75%

IV. Results

A. Experiment One

To compare the set associativity vs miss rate, five different versions of the simulation were run. The total size of the cache was set to 32 KiB, with a block size of 64 Bytes. The associativity was set to direct-mapped, 2-way, 4-way, 8-way, and fully associative. The trace file was random values generated with "trace_gen.py."

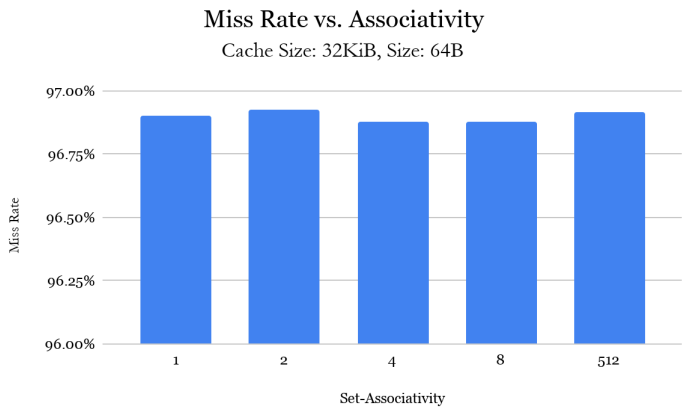

Fig. 1. Bar graph of set-associativity vs miss rate. All values around around 96% with slight variations

As one can see, the association between the cache sets does not change the miss rate. This is because in a random set of numbers, a hit is very unlikely, meaning that even with more associativity, it does not change the miss rate.

B. Experiment Two

In this experiment, two different trace files were used. One was a linear increment of address values, while the other was a random assortment of values. Both of these used 64 sets, 8-way associativity, and a size of 64 bytes.

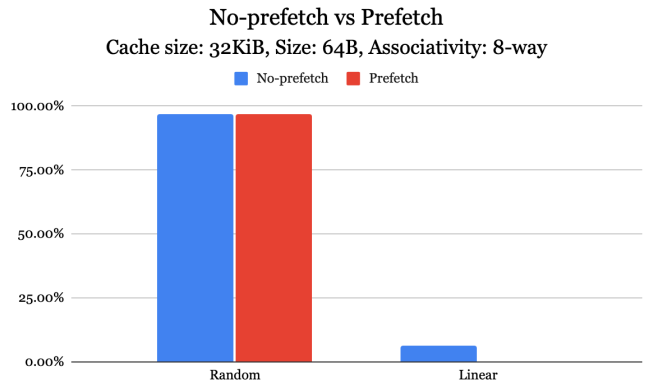

Fig. 2. Bar graph of no-prefetch and prefetch. Linear with prefetch had a miss rate of 0.001%, nearly zero.

The prefetching option has hardly any effect on the random miss rate. Since values are so random, the prefetching does virtually nothing, since the chance of the cache accessing the next block is very slim. In fact, this is known as polluting the cache, and the prefetch option for the random access actually has a lower number of hits.

In the linear simulation, the prefetcher makes it so that the very first access is a miss, and all the rest are hits. This is because the next value will always get loaded into the cache when the current value is a hit.

V. Conclusion

When implementing a caching system, there are many parameters that can be changed and manipulated in order to achieve better performance. In the course of testing our simulation and gathering data we can clearly see the effect of a variety of scenarios.

Primarily it is noted that enabling prefetching when caching linear addresses has the largest effect of any optimization, reducing the miss count to one. However for random memory addresses this optimization is almost negligible and in some cases has a negative impact as it can fill up the cache with adjacent values that are not being accessed causing more misses.

By manipulating the size of the cache, the number of sets, blocks, and size of those blocks, we see a variety of results, most with diminishing returns. While increasing the cache size can have a positive effect, eventually it can become too large and the likelihood of a hit begins to decrease. Similarly effects are found with the number of sets and the block size.

References

[1] Patterson, David A, and John L Hennessy. *Computer Organization and Design : The Hardware/Software Interface:* ARM edition. Cambridge, MA: Morgan Kaufmann, 2018. [E-Book]. Available: UTK Online Library.